

TECH

LIVE-CODING

JAVASCRIPT / ALGORITHMS **TYPESCRIPT** REACT

TYPESCRIPT

20 ЗАДАЧ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК [type-challenges](#) / [ts-handbook](#)

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	PARTIAL K OMIT Q1 · Q2 · Q3	REQUIRED	READONLY	PIC	PARTIAL K OMIT	REQUIRED	READONLY	PIC
ДЕНЬ 2 ТЯЖЁЛЫЙ	DEEP PARTIAL Q4 · Q5	DEEP READONLY			DEEP PARTIAL	DEEP READONLY		
ДЕНЬ 3 СРЕДНИЙ	AWAITED PARAMETERS Q6 · Q7 · Q8	PROMISE TYPE	RETURN TYPE	AWAITED PARAMETERS	PROMISE TYPE	RETURN TYPE		
ДЕНЬ 4 ТЯЖЁЛЫЙ	UNION TO INTERSECT KEYS Q9 · Q10 · Q11	TUPLE TO UNION		UNION TO INTERSECT KEYS	TUPLE TO UNION			
ДЕНЬ 5 ТЯЖЁЛЫЙ	GET TYPE BY PATH Q12 · Q13	SET TYPE BY PATH		GET TYPE BY PATH	SET TYPE BY PATH			
ДЕНЬ 6 ТЯЖЁЛЫЙ	BRANDED EVENT Q14 · Q15 · Q16	NOMINAL	OPAQUE	TYPED	BRANDED EVENT	NOMINAL	OPAQUE	TYPED
ДЕНЬ 7 ТЯЖЁЛЫЙ	BUILDER Q17 · Q18 · Q19 · Q20	REDUCER	DISCRIMINATED FN		BUILDER	REDUCER	DISCRIMINATED FN	

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ

ДЕНЬ 7 – БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

PARTIAL**REQUIRED****READONLY****PICK****OMIT**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

PARTIAL REQUIRED READONLY PICK OMIT

ВОПРОСЫ ДНЯ

Q01 Реализуй MyPartial<T>

Q02 Реализуй MyRequired<T>

Q03 Реализуй MyPick<T, K>

Реализуй MyPartial<T>

ОПИСАНИЕ

Делает все поля optional. Mapped type с ?: модификатором.

КАК РАБОТАЕТ

- 01 type MyPartial<T> = { [K in keyof T]?: T[K] }.
 - 02 K in keyof – итерация.
 - 03 ? делает optional.
 - 04 Применять с care – потеря строгости.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Form state.
 - 02 Update DTOs.
 - 03 Patch operations.
-

ПОДВОДНЫЕ КАМНИ

- 01 Required и Partial – взаимные.
 - 02 DeepPartial для nested.
-

МОГУТ ДОСПРОСИТЬ

- 01 Чем Required от NonNullable отличается?
 - 02 Что такое mapped types?
-

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/mapped-types.html>

ВОПРОС Q02

Реализуй MyRequired<T>**ОПИСАНИЕ**

Делает все optional обязательными. -? убирает optional.

КАК РАБОТАЕТ

- 01 type MyRequired<T> = { [K in keyof T]-?: T[K] }.
- 02 -? – remove optional.
- 03 Также есть -readonly.
- 04 Не убирает undefined из union.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Form validated state.
- 02 Required DTO.

ПОДВОДНЫЕ КАМНИ

- 01 undefined остаётся в значении – нужен NonNullable<T[K]>.
- 02 -readonly симметричен.

МОГУТ ДОСПРОСИТЬ

- 01 Что делает -readonly?
- 02 Чем NonNullable отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html#requiredtype>

Реализуй MyPick<T, K>

ОПИСАНИЕ

Берёт только указанные ключи. Constrain K extends keyof T.

КАК РАБОТАЕТ

- 01 type MyPick<T, K extends keyof T> = { [P in K]: T[P] }.
 - 02 Constrain K – only valid keys.
 - 03 P in K – итерация.
 - 04 Omit – обратная операция.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 DTO subsetting.
 - 02 Component props slice.
-

ПОДВОДНЫЕ КАМНИ

- 01 Без constraint – K может быть any string.
 - 02 Pick<T, never> = {}.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Omit?
 - 02 Чем Extract отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html#picktype-keys>

ДЕНЬ 2 НАГРУЗКА ТЯЖЁЛЫЙ 2 ВОПРОСА

DEEP PARTIAL DEEP READONLY

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DEEP PARTIAL DEEP READONLY

ВОПРОСЫ ДНЯ

Q04 Реализуй DeepPartial<T>

Q05 Реализуй DeepReadonly<T>

Реализуй DeepPartial<T>

ОПИСАНИЕ

Рекурсивный Partial. Все nested objects – тоже Partial.

КАК РАБОТАЕТ

- 01 type DeepPartial<T> = T extends object ? { [K in keyof T]?: DeepPartial<T[K]> } : T.
- 02 Conditional + recursion.
- 03 Edge: arrays, functions, primitives.
- 04 Спец-case для Date, Map, Set.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Form state nested.
- 02 Settings updates.
- 03 API patches.

ПОДВОДНЫЕ КАМНИ

- 01 Function – тоже object – нужно exclude.
- 02 Array<T> → Array<DeepPartial<T>>.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое recursive type?
- 02 Чем DeepPartial от Partial отличается?

ПОЛНАЯ СТАТЬЯ

<https://github.com/krzkaczor/ts-essentials>

ВОПРОС Q05

Реализуй DeepReadonly<T>

ОПИСАНИЕ

Рекурсивный Readonly. Frozen на всех уровнях.

КАК РАБОТАЕТ

- 01 type DeepReadonly<T> = { readonly [K in keyof T]: DeepReadonly<T[K]> }.
- 02 Conditional для primitives.
- 03 Array<T> → ReadonlyArray<DeepReadonly<T>>.
- 04 Функции — оставляем как есть.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Immutable state.
- 02 Redux store types.

ПОДВОДНЫЕ КАМНИ

- 01 DeepReadonly<Function> — strange.
- 02 ReadonlyArray не убирает push в runtime.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Object.freeze?
- 02 Чем readonly от const отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html#readonlytype>

AWAITED

PROMISE TYPE

RETURN TYPE

PARAMETERS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

AWAITED PROMISE TYPE RETURN TYPE PARAMETERS

ВОПРОСЫ ДНЯ

Q06 Реализуй MyAwaited<T>

Q07 Реализуй MyReturnType<T>

Q08 Реализуй MyParameters<T>

ВОПРОС Q06

Реализуй MyAwaited<T>

ОПИСАНИЕ

Извлекает тип из Promise<T>. Если nested Promise – recurse.

КАК РАБОТАЕТ

- 01 type MyAwaited<T> = T extends Promise<infer U> ? MyAwaited<U> : T.
- 02 infer U – захватывает inner type.
- 03 Recursion для nested Promise.
- 04 Edge: T is not Promise.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 async return types.
- 02 Library typings.

ПОДВОДНЫЕ КАМНИ

- 01 Promise<Promise<T>> – нужен recurse.
- 02 Awaited есть в lib встроенный.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое infer?
- 02 Чем conditional от mapped отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html#awaitedtype>

Реализуй MyReturnType<T>

ОПИСАНИЕ

Извлекает return type функции.

КАК РАБОТАЕТ

- 01 `type MyReturnType<T> = T extends (...args: any[]) => infer R ? R : never.`
- 02 `infer R` – return type.
- 03 `never` – fallback.
- 04 Не работает для overloads – берёт last.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Library typings.
- 02 Hook return types.
- 03 Generic helpers.

ПОДВОДНЫЕ КАМНИ

- 01 Overloads – берёт last signature.
- 02 Default never – может быть проблема.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое overload resolution?
- 02 Чем Parameters от ReturnType отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html#returntypetype>

ВОПРОС Q08

Реализуй MyParameters<T>

ОПИСАНИЕ

Извлекает tuple параметров функции.

КАК РАБОТАЕТ

- 01 type MyParameters<T> = T extends (...args: infer P) => any ? P : never.
- 02 P захватывает все параметры как tuple.
- 03 Spread argument для forwarding.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Forwarding params.
- 02 Generic wrappers.
- 03 HOC typings.

ПОДВОДНЫЕ КАМНИ

- 01 Generic functions – нужны constraints.
- 02 Overloads – last signature.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое ConstructorParameters?
- 02 Чем tuple от array отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/utility-types.html#parameterstype>

UNION TO INTERSECT

TUPLE TO UNION

KEYS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

UNION TO INTERSECT TUPLE TO UNION KEYS

ВОПРОСЫ ДНЯ

Q09 Реализуй `UnionToIntersection<U>`Q10 Реализуй `TupleToUnion<T>`Q11 Реализуй `ObjectKeys<T>`

ВОПРОС Q09

Реализуй UnionToIntersection<U>

ОПИСАНИЕ

Преобразует $A \mid B$ в $A \& B$. Через distributive conditional + contravariance.

КАК РАБОТАЕТ

- 01 `type UnionToIntersection<U> = (U extends any ? (k: U) => void : never) extends (k: infer I) => void ? I : never.`
- 02 Contravariance в parameter position.
- 03 Distributive over union.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mixin types.
- 02 Plugin merging.

ПОДВОДНЫЕ КАМНИ

- 01 Сложно объяснить — не используй без необходимости.
- 02 Может break'ять inference.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое contravariance?
- 02 Чем distributive conditional полезен?

ПОЛНАЯ СТАТЬЯ

<https://github.com/type-challenges/type-challenges>

Реализуй TupleToUnion<T>

ОПИСАНИЕ

$[A, B, C] \rightarrow A \mid B \mid C$. `T[number]` – простой trick.

КАК РАБОТАЕТ

- 01 `type TupleToUnion<T extends any[]> = T[number]`.
 - 02 Index access с number.
 - 03 Edge: пустой tuple $\rightarrow$ never.
 - 04 Полезно для enum-like.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Const arrays $\rightarrow$ union types.
 - 02 Validation lists.
-

ПОДВОДНЫЕ КАМНИ

- 01 `T[number]` requires array.
 - 02 readonly tuples – readonly `T[number]`.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое const assertion?
 - 02 Чем tuple от array отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/objects.html#tuple-types>

ВОПРОС Q11

Реализуй ObjectKeys<T>

ОПИСАНИЕ

Возвращает union строковых keys объекта. Аналог keyof T но строгий.

КАК РАБОТАЕТ

- 01 type ObjectKeys<T> = keyof T.
- 02 Если все keys – string: keyof T extends string ? keyof T : never.
- 03 Object.keys в runtime теряет точность.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Pick fields.
- 02 Form field iteration.

ПОДВОДНЫЕ КАМНИ

- 01 keyof возвращает string|number|symbol – нужен extract<keyof T, string>.
- 02 Object.keys runtime → string[].

МОГУТ ДОСПРОСИТЬ

- 01 Что такое keyof?
- 02 Чем Object.keys от for-in отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/keyof-types.html>

GET TYPE BY PATH**SET TYPE BY PATH**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

GET TYPE BY PATH SET TYPE BY PATH

ВОПРОСЫ ДНЯ

Q12 Реализуй `Get<T, Path>` для dot-path accessQ13 Реализуй `Set<T, Path, Value>` для dot-path mutation

ВОПРОС Q12

Реализуй Get<T, Path> для dot-path access

ОПИСАНИЕ

Get<{a:{b:1}}, 'a.b'> = 1. Recursive split path и index access.

КАК РАБОТАЕТ

- 01 Conditional + template literal.
- 02 T extends infer split & template literal.
- 03 Recurse: Get<T[Head], Tail>.
- 04 Edge: path не существует → never.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Lodash.get с типами.
- 02 Form path validation.

ПОДВОДНЫЕ КАМНИ

- 01 Сложная дыра в TS.
- 02 Path не существует – never (а не undefined).

МОГУТ ДОСПРОСИТЬ

- 01 Что такое template literal types?
- 02 Чем infer полезен?

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/template-literal-types.html>

Реализуй Set<T, Path, Value> для dot-path mutation

ОПИСАНИЕ

Set<{a:{b:1}}, 'a.b', 2> = {a:{b:2}}. Reconstructive type.

КАК РАБОТАЕТ

- 01 Recursive: split path.
- 02 Recurse Set<T[Head], Tail, Value>.
- 03 Mapped type для пересборки.
- 04 Edge: path не существует – добавь.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Type-safe lodash.set.
- 02 Form patches.

ПОДВОДНЫЕ КАМНИ

- 01 Очень сложно – limit recursion.
- 02 Неоднозначность для optional полей.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое recursion limit в TS?
- 02 Чем set от update отличается?

ПОЛНАЯ СТАТЬЯ

<https://github.com/type-challenges/type-challenges>

ДЕНЬ 6 НАГРУЗКА ТЯЖЁЛЫЙ 3 ВОПРОСА

BRANDED**NOMINAL****OPAQUE****TYPED EVENT**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BRANDED NOMINAL OPAQUE TYPED EVENT

ВОПРОСЫ ДНЯ

Q14 Реализуй Branded<T, Brand>

Q15 Реализуй Opaque<T, Token>

Q16 Реализуй TypedEventEmitter<Events>

Реализуй Branded<T, Brand>

ОПИСАНИЕ

Nominal typing через intersection с unique tag. UserId vs string.

КАК РАБОТАЕТ

- 01 type Brand<T, B> = T & { readonly __brand: B }.
 - 02 Casting через as: const id = '1' as UserId.
 - 03 Compile-time only – runtime: string.
 - 04 Constructor-функция для validation.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 IDs (UserId, OrderId).
 - 02 Money amounts (USD, EUR).
 - 03 Sanitised strings.
-

ПОДВОДНЫЕ КАМНИ

- 01 Brand доступен в runtime – нет.
 - 02 Без factory – easy to bypass.
-

МОГУТ ДОСПРОСИТЬ

- 01 Чем nominal от structural typing отличается?
 - 02 Что такое opaque types?
-

ПОЛНАЯ СТАТЬЯ

<https://egghead.io/blog/using-branded-types-in-typescript>

ВОПРОС Q15

Реализуй Opaque<T, Token>

ОПИСАНИЕ

Аналог branded но через unique symbol. Невозможно случайно создать значение.

КАК РАБОТАЕТ

- 01 declare const tag: unique symbol.
- 02 type Opaque<T, K> = T & { readonly [tag]: K }.
- 03 Только factory может создать.
- 04 Runtime – это просто T.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Sensitive IDs.
- 02 Validated inputs.
- 03 Currency.

ПОДВОДНЫЕ КАМНИ

- 01 Symbol can't be exported normally.
- 02 Strict null check важен.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое unique symbol?
- 02 Чем opaque от branded отличается?

ПОЛНАЯ СТАТЬЯ

<https://github.com/type-fest/type-fest>

Реализуй TypedEventEmitter<Events>

ОПИСАНИЕ

EventEmitter с типами событий. on(event, handler) – type-checked.

КАК РАБОТАЕТ

```
01 interface Events { 'click': MouseEvent; 'change': string }.
02 on<K extends keyof Events>(e: K, h: (p: Events[K]) => void).
03 emit<K>(e: K, p: Events[K]).
04 Generic K extends keyof Events.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Domain events.
02 Plugin system.
03 Custom event bus.
```

ПОДВОДНЫЕ КАМНИ

```
01 string events – потеря типизации.
02 Без conditional – payload может быть optional.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое keyof + index access?
02 Чем discriminated union полезен?
```

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

ДЕНЬ 7 НАГРУЗКА ТЯЖЁЛЫЙ 4 ВОПРОСА

BUILDER REDUCER DISCRIMINATED FN

ЧТО УЧИМ В ЭТОТ ДЕНЬ

BUILDER REDUCER DISCRIMINATED FN

ВОПРОСЫ ДНЯ

- Q17 Реализуй Builder<T> с fluent API
- Q18 Реализуй type-safe reducer с discriminated union
- Q19 Реализуй generic-функцию pluck<T, K>
- Q20 Реализуй ExhaustiveCheck для switch

Реализуй Builder<T> с fluent API

ОПИСАНИЕ

Chain методов set возвращающих this с уточнённым типом. Final build() требует все required.

КАК РАБОТАЕТ

```
01 class Builder<T = {}>.  
02   with<K, V>(k: K, v: V): Builder<T & {[P in K]: V}>.  
03   Возвращай new Builder через cast.  
04   build(): T.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Config builders.  
02 Test fixtures.
```

ПОДВОДНЫЕ КАМНИ

```
01 Mutation vs immutable – важно для chain.  
02 Required check на build().
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое fluent API?  
02 Чем builder от factory отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://refactoring.guru/design-patterns/builder>

ВОПРОС Q18

Реализуй type-safe reducer с discriminated union

ОПИСАНИЕ

Action как `{type, payload}`. Reducer switch по type – narrowed payload.

КАК РАБОТАЕТ

- 01 `type Action = {type: 'add'; payload: number} | {type: 'reset'}.`
- 02 `switch (action.type)` – TS narrows payload.
- 03 exhaustiveness check через `never`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Redux.
- 02 `useReducer`.
- 03 State machines.

ПОДВОДНЫЕ КАМНИ

- 01 `string actions` – потеря типизации.
- 02 default case без `never` – silent miss.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое exhaustive check?
- 02 Чем `switch` от `if-else` для `union` отличается?

ПОЛНАЯ СТАТЬЯ

<https://redux.js.org/usage/usage-with-typescript>

Реализуй generic-функцию `pluck<T, K>`

ОПИСАНИЕ

`pluck(arr, key) → arr.map(x => x[key])`. Type-safe key и return type.

КАК РАБОТАЕТ

```
01 function pluck<T, K extends keyof T>(arr: T[], key: K): T[K][].  
02   return arr.map(x => x[key]).  
03   K extends keyof – only valid keys.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Lodash-like utilities.  
02 DTO transforms.
```

ПОДВОДНЫЕ КАМНИ

```
01 any[] вместо T[] – потеря типизации.  
02 K без constraint – runtime errors.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое keyof?  
02 Чем pluck от map отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/generics.html>

ВОПРОС Q20

Реализуй ExhaustiveCheck для switch

ОПИСАНИЕ

function assertNever(x: never): never. Compile-error если case пропущен.

КАК РАБОТАЕТ

```
01 function assertNever(x: never): never { throw new Error('Unexpected: ' + x) }.
02 default: assertNever(action) – TS error если есть необработанный case.
03 Pattern для discriminated union.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 State machines.
02 Action handlers.
03 Type guards.
```

ПОДВОДНЫЕ КАМНИ

```
01 Без default – silent miss.
02 throw в production – bad UX.
```

МОГУТ ДОСПРОСИТЬ

```
01 Что такое never type?
02 Чем exhaustive switch от if-else отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://www.typescriptlang.org/docs/handbook/2/narrowing.html#exhaustiveness-checking>

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

☐ Реализовал MyPartial / Required / Pick / Omit

☐ DeepPartial / DeepReadonly с recursion

☐ Awaited / ReturnType / Parameters через infer

☐ UnionToIntersection / TupleToUnion

☐ Get / Set по dot-path с template literal

☐ Branded / Opaque types для domain IDs

☐ TypedEventEmitter с keyof Events

☐ Exhaustive check через assertNever

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru